CSE 314 — Operating System Sessional
Online on xv6 Scheduling · Section B2

Budget Epochs

30 minutes. Fresh xv6 checkout, CPUS := 1.

The Idea

Replace the round-robin scheduler with one that gives each process a budget of selections to run, then refills the budgets at the start of a new epoch.

- Every process starts with a full budget: BUDGET_UNIT × priority.
- Pick the RUNNABLE process with the largest positive remaining budget; break ties by process-table order. A process with budget 0 is ineligible.
- Subtract one unit from the selected process's budget before running it. It runs until it yields, blocks, or exits; even an early block costs that one unit.
- If at least one process is RUNNABLE and all RUNNABLE processes have budget 0, start a new epoch: refill every process's budget to BUDGET_UNIT × its own priority, including sleeping processes. Then select again.

If no process is RUNNABLE, wait for work without refilling. Sleeping processes otherwise keep their remaining budgets unchanged. Budget counts selections, not timer ticks; no timer-handler changes or explicit queues are required.

Specification Parameters

Define the following macros in kernel/param.h:

Macro	Value	Meaning
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	5	Priority every process starts with.
`BUDGET_UNIT`	2	Selections granted per unit of priority.

System Calls

```
int setpriority(int priority);
int getpriority(void);
```

setpriority sets the caller's priority, refills its budget, and returns 0, even if the priority is unchanged. Invalid input resets priority to DEFAULT_PRIORITY, refills the budget, and returns −1. getpriority returns the current priority. A fork() child inherits its parent's priority but starts with a full budget, not the parent's remaining budget.

The syscall bodies are supplied in syspriority.c; append them to kernel/sysproc.c. Add int priority and int budget to struct proc, initialize them on allocation and fork, and wire the calls through kernel/syscall.h, kernel/syscall.c, user/usys.pl and user/user.h using unused syscall numbers.

Logging

Keep a toggleable log line for every selection, showing the budget before subtracting one — a global flag or #define that you flip and rebuild is fine:

```
PICK: Process 5 (testloop) runs with budget 8 at priority 4
```

Test Program

Use the attached testloop.c as-is: save it to user/testloop.c and add $U/_testloop to UPROGS in the Makefile. Calling syntax: testloop <iterations> <priority>.

Sample Session

Three testloop processes, equal work, different priorities, launched one line at a time (scheduler logging off). The & runs each process in the background so the next can be launched while it is still running:

```
$ testloop 60 8 &
PID 4: starting 60 iterations at time 8, priority 8
$ testloop 60 4 &
PID 6: starting 60 iterations at time 20, priority 4
$ testloop 60 2 &
PID 8: starting 60 iterations at time 42, priority 2
PID 4: finished at time 64
PID 6: finished at time 101
PID 8: finished at time 124
```

This session was captured under QEMU with CPUS := 1 using the supplied testloop.c and a reference implementation. PIDs, timestamps and completion order vary; exact timings are not part of evaluation. Budgets grant selections, not a guaranteed amount of CPU time.

For a deterministic check, suppose only P1 and P2 are continuously runnable, in that table order, with priorities 2 and 1 and full budgets 4 and 2. The first six selections are P1, P1, P1, P2, P1, P2. Both budgets are then 0; refill them to 4 and 2 before the seventh selection, which picks P1.

Submission

Run these commands from the xv6 checkout. Replace STUDENT_ID with your student ID and submit the patch containing your scheduling changes and test program.

```
git add --all
git diff HEAD > ../STUDENT_ID.patch
```